Computers & Security 166 (2026) 104884

Contents lists available at ScienceDirect

Computers & Security

journal homepage: www.elsevier.com/locate/cose

Full Length Article

ROParser: A high-efficient framework for return-oriented programming deobfuscation

Haoran Liu [a], Tieming Liu [a], Rui Chang [b], Jian Lin [a,*], Zuozheng Zhou [a], Jing Jing [a]

[a] Information Engineering University, Zhengzhou, 450001, China
[b] Zhejiang University, Hangzhou, 310027, China

ARTICLE INFO

Keywords:
Deobfuscation
ROP
Symbolic execution
LLM
Code reuse attack

ABSTRACT

Return-Oriented Programming (ROP) is a sophisticated vulnerability exploitation technique that bypasses modern security defenses. ROP obfuscation leverages ROP to create complex control flows, making program logic extremely difficult to decipher and effectively hindering reverse engineering analysis. However, existing ROP analysis methods primarily focus on analyzing ROP exploitation payloads. They are inefficient when handling complex ROP obfuscations that involve robust control flow obfuscation, custom stack operations, and redundant instructions.

In this paper, we systematically analyze the limitations of state-of-the-art ROP analysis methods and address three key challenges: robust control flow recovery, custom stack operation instruction recovery, and redundant instruction optimization. Building on these insights, we present `ROParser`, a novel ROP deobfuscation framework. `ROParser` reconstructs a complete Control Flow Graph (CFG) through localized symbolic execution and reduced value-set analysis, recovers custom stack operation instructions using few-shot learning with a Large Language Model (LLM), and optimizes redundant instructions through semantics-guided redundant code elimination.

Comprehensive evaluations across diverse datasets demonstrate that `ROParser` achieves 97.97% CFG recovery accuracy, improving from the baseline of 91.38%. Even under compound obfuscation settings that combine multiple state-of-the-art techniques, `ROParser` still recovers nearly 90% of the original CFG. Furthermore, the framework successfully optimizes 99.16% of custom stack operations and eliminates 22.84% of redundant code. These results highlight the potential of `ROParser` for practical applications in ROP deobfuscation and ROP-based malicious payloads analysis.

1. Introduction

The widespread adoption of computer systems and software applications has fueled an ongoing security arms race between attackers and defenders (De Sutter et al., 2024). Return-Oriented Programming (ROP) (Shacham, 2007) plays a unique and crucial role in software security. As a type of code reuse attack (Bletsch et al., 2011; Schuster et al., 2015), ROP extracts small code fragments (gadgets) from binaries loaded by the operating system to construct attack instructions, allowing attackers to bypass the mechanism *data execution protection (DEP)* of most modern operating systems. Due to its powerful attack capabilities, ROP has become one of the most prevalent attack methods, widely used for binary vulnerability exploitation and malicious code injection.

Recently, research has demonstrated the feasibility of ROP obfuscation. Mu et al. (2018) proposed ROPOB, a binary-level ROP obfuscation technique. Borrello et al. (2021) introduced Raindrop, a method that transforms a program into obfuscated ROP chains while rewriting stack operation instructions. De Pasquale et al. (2023) developed a framework combining opaque predicates and instruction hiding to enhance ROP obfuscation. ROP chains have been extended beyond typical exploitation to applications like antivirus evasion (Ntantogian et al., 2019), program steganography (Lu et al., 2014), and code integrity verification (Andriesse et al., 2015).

ROP-based obfuscation fundamentally alters program execution by constructing ROP chains from return-oriented gadgets, fragmenting control flow and semantics. This fragmentation, when combined with the enhanced obfuscation strategy, makes static analysis, symbolic execution, and taint analysis ineffective, presenting substantial challenges for reverse engineers. Although ROP-based obfuscation introduces some implementation complexity and runtime overhead, it significantly increases analysis complexity, making it a potent tool for scenarios that require strong protection.

* Corresponding author.
E-mail addresses: Cherest_San@outlook.com (H. Liu), fxliutm@163.com (T. Liu), crix1021@zju.edu.cn (R. Chang), ling_pro@163.com (J. Lin), 2122817150@qq.com (Z. Zhou), jingjing_cs@hotmail.com (J. Jing).

https://doi.org/10.1016/j.cose.2026.104884
Received 14 October 2025; Received in revised form 4 February 2026; Accepted 28 February 2026
Available online 6 March 2026

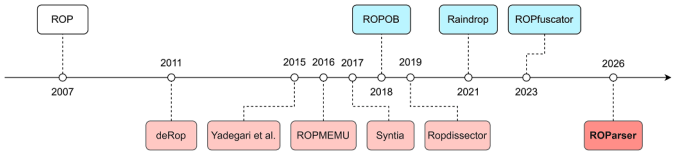


Fig. 1. Timeline of ROP techniques.

ROP obfuscation can be broadly classified into two categories. First, binary-rewriting-based obfuscation (e.g., ROPOB and Raindrop) transforms original instructions into ROP gadgets, converting the entire program logic into the ROP chain. Second, library-reuse-based obfuscation (e.g., ROPfuscator) replaces selected code regions with ROP chains sourced from existing library gadgets, while rewriting both direct and indirect control flow transfers. Although these methods differ in their construction strategies, both fundamentally obscure control flow and fragment instruction semantics, complicating automated deobfuscation efforts.

Given the prevalence of ROP, various methods have been proposed for analyzing ROP chains to detect payloads and counter ROP-based malware. Early works such as deRop (Lu et al., 2011) used static analysis to recover control flow and translate ROP chains into shellcode. ROPMEMU (Graziano et al., 2016) employed emulation to improve branch coverage by manipulating EFLAGS. ROPdissector (D'Elia et al., 2019) combined lightweight emulation and taint analysis to extend control flow recovery.

Beyond these ROP-specific techniques, influential works by Yadegari et al. (2015) and Syntia (Blazytko et al., 2017) have provided general deobfuscation frameworks that address ROP, highlighting its significance in broader deobfuscation. However, these frameworks are not designed exclusively for ROP deobfuscation and often fail to address the aggressive, specific challenges posed by ROP obfuscation.

Fig. 1 displays the chronological development of ROP analysis methods (in red) and ROP obfuscation techniques (in blue). Existing ROP deobfuscation research exhibits an apparent asymmetry: while obfuscation techniques have rapidly evolved toward aggressive, program-level transformations, deobfuscation methods remain primarily focused on analyzing simple ROP exploit payloads. Furthermore, traditional static analysis methods, such as symbolic execution and taint analysis, fail to recover the complete control flow and suffer from severe state explosion. Based on our review of existing research, we identify three key challenges in ROP deobfuscation:

Challenge 1 (C1): Robust Control Flow Recovery. Modern ROP obfuscators combine multiple control flow transformations-such as opaque-encoded branch, state explosion, anti-bruteforce, gadget confusion, and indirect branch. These techniques interact to obscure actual branch targets, inflate symbolic execution states, and break conventional gadget tracking. Existing deobfuscation approaches cannot deal with these aggressive obfuscation strategies, making the recovery of an accurate CFG under compound obfuscation a fundamental challenge.

Challenge 2 (C2): Custom Stack Operation Instruction Recovery. Binary-rewriting-based ROP Obfuscation can replace the stack operation instructions with custom indirect stack addressing operations to preserve the integrity of the native stack during obfuscation. Existing deobfuscation techniques cannot recover such custom stack operation instructions.

Challenge 3 (C3): Redundant Instruction Optimization. Library-reuse-based ROP obfuscation often extracts gadgets from libraries, which may include instructions that are semantically irrelevant to program execution. Additionally, the obfuscation technique decomposes the instruction into multiple sub-instructions and an aggressive obfuscation configuration introduces redundant control flow. Existing deobfuscation approaches lack effective techniques to optimize such redundant instructions.

To address these challenges, we implement an efficient framework for ROP-based program deobfuscation, called `ROParser`, which presents three techniques as follows.

- **Robust Control Flow Recovery.** To solve C1, `ROParser` implements *Localized Symbolic Execution* to reconstruct the Control Flow Graph (CFG) of ROP-obfuscated programs and implements *Reduced Value-set Analysis* to explore complete branches, including those caused by aggressive obfuscation techniques.
- **LLM-Based Few-shot Learning.** To solve C2, `ROParser` applies few-shot learning with LLM to identify and recover custom stack operation instructions. This is accomplished by leveraging the powerful semantic analysis capabilities of LLMs.
- **Semantics-Guided Redundant Code Elimination.** To solve C3, `ROParser` introduces a multi-granularity optimization strategy targeting both instruction and basic-block redundancy. It combines instruction-level constant propagation and liveness analysis to simplify gadgets, and a block-level pruning algorithm that eliminates semantically irrelevant control flow based on global side-effect analysis.

Our evaluation demonstrates that `ROParser` effectively deobfuscates ROP-based obfuscated programs and effectively addresses the above-mentioned challenges. In summary, we make the following contributions.

- **Systematic Study:** We systematically analyzed the existing ROP analysis methodologies and their limitations. We conducted a comprehensive threat model study of modern ROP obfuscation that hinders CFG reconstruction and instruction semantics recovery. We revealed three key challenges that significantly impact the effectiveness of current ROP analysis approaches: *Robust Control Flow Recovery*, *Custom Stack Operation Instruction Recovery*, and *Redundant Instruction Optimization*.
- **Framework Implementation:** We present `ROParser`, an ROP deobfuscation framework. The framework reconstructs the CFG using *Localized Symbolic Execution*, handles branches using *Reduced Value-set Analysis*, recovers the custom stack operation instruction using a few-shot learning approach with an LLM, and optimizes instructions using semantics-guided redundant code elimination. Our code will be open-sourced on GitHub at https://github.com/Cherest-San/ROParser.
- **Comprehensive Evaluation:** We conducted a comprehensive evaluation of `ROParser` using three datasets to demonstrate the effectiveness of `ROParser`. Our method increases CFG recovery accuracy from 91.38% to 97.97%, surpassing the state-of-the-art tool. Even under the most aggressive obfuscation settings, `ROParser` still recovers nearly 90% of the original CFG. Meanwhile, unlike existing tools that do not support instruction optimization, `ROParser` achieves superior performance by optimizing 99.16% of custom stack instructions and eliminating 22.84% of redundant instructions.

2. Background

In this section, we introduce the technical background of our research, including the ROP technique, its application to code obfuscation, and existing ROP deobfuscation techniques.

Return-oriented Programming (ROP). ROP is an advanced memory exploitation technique that can bypass the defenses of modern operating systems (Shacham, 2007; Angelini et al., 2018; Zhong et al., 2024). The fundamental principle of ROP is the construction of small code segments called gadgets (Follner et al., 2016) that end with a *RET* instruction. These gadgets are often found in various dynamic link libraries. They can be strategically grouped on the target stack to execute arbitrary code, effectively bypassing *Data Execution Protection (DEP)* mechanisms. ROP has many applications and can be used across various target

platforms, including X86, ARM, and RISC-V (Taki et al., 2024; Cloosters et al., 2022).

ROP Obfuscation. ROP chains, comprising sequences of gadgets, contain complete semantic information and execution logic. This characteristic makes ROP suitable for control flow obfuscation, where normal control flow is transformed into ROP chains, thereby significantly increasing the complexity of reverse engineering (Banescu et al., 2016). The effectiveness of ROP obfuscation can also be further enhanced by complementary techniques such as opaque predicates (Collberg et al., 1998) and instruction hiding (De Pasquale et al., 2023).

ROP obfuscation methods are divided into two categories. The first is binary-rewriting-based ROP obfuscation. ROPOB (Mu et al., 2018) is a method of obfuscation using ROP against basic blocks by changing direct control flow to indirect control flow. Borrello et al. proposed Raindrop (Borrello et al., 2021), which automatically constructs the ROP chain from the target program using a binary rewrite method and refactors all native stack operations into custom stack operation instructions.

The second category is library-reuse-based ROP obfuscation. ROPfuscator (De Pasquale et al., 2023) proposed an ROP obfuscation method, which automates the search for available gadgets, applies dynamic symbolic addressing against the ASLR mechanism, and adds opaque predicates and command hiding mechanisms to counter traditional deobfuscation mechanisms.

ROP Deobfuscation. Current ROP deobfuscation approaches (Udupa et al., 2005) are mainly divided into static and dynamic analysis. The methods aim to recover the original control flow from the ROP chains.

Static Analysis. Static analysis methods identify the sequential relationships between gadgets without executing the code. Although this approach offers simplicity and safety by avoiding code execution, it faces challenges in handling dynamically generated code and complex control flows.

deRop (Lu et al., 2011), one of the first ROP deobfuscation tools, converts ROP chains to traditional shellcode through static analysis, tracks control flow changes by statically locating the ESP, and introduces a heuristic function call identification method.

Yadegari et al. (2015) attempts to design a generic deobfuscation method using symbolic execution and taint analysis, analyzes control dependencies, and recovers control flow through data flow analysis.

Syntia (Blazytko et al., 2017) attempts to analyze the semantics of obfuscation programs using Monte Carlo Tree Search (MCTS) and adapts to different obfuscation methods by adjusting the parameters.

Dynamic Analysis. Dynamic analysis methods capture runtime behavior by simulating or executing ROP code. They use debuggers or emulators to track instruction execution, register value changes, and memory access patterns, thereby reconstructing the control flow graph of ROP chains. The advantage of dynamic analysis is the high accuracy of the CFG, and the disadvantage is the need for accurate execution or simulation.

ROPMEMU (Graziano et al., 2016) is an emulation-based framework for multipath ROP deobfuscation, which recovers control flow through emulation tracking and shadow memory while analyzing conditional branches by manipulating EFLAGS to explore all possible execution paths.

ROPdissector (D'Elia et al., 2019) represents the latest deobfuscation method. It employs lightweight emulation and shadow memory techniques to assist in the static analysis of ROP chains, capturing all possible execution paths via multipath exploration to recover complete control flow information.

3. Threats and challenges

In this section, we present a comprehensive analysis of the obstacles facing ROP deobfuscation. We first define a rigorous threat model that characterizes the capabilities of a sophisticated adversary employing state-of-the-art obfuscation techniques. Subsequently, we examine the limitations of existing analysis frameworks when confronting these threats, highlighting three critical challenges. Finally, we outline the motivation and design goals of `ROParser` to address these gaps.

3.1. Threat model

We consider a capable adversary employing ROP obfuscation, whose goal is to disrupt control flow extraction and instruction semantics recovery by utilizing transformations that complicate static analysis and symbolic execution. Our threat model characterizes widely employed obfuscation techniques at both the control flow and instruction levels, including those implemented in state-of-the-art ROP obfuscators. Importantly, the obfuscation techniques in our threat model are not assumed to appear in isolation. Instead, they may be combined arbitrarily or applied simultaneously, thereby compounding the difficulty of analysis.

3.1.1. Control flow level

T1. Opaque-Encoded Branch. Obfuscators hide branch targets by encoding part of each branch offset into a periodic array of opaque values. Only a subset of array entries is meaningful, and aliasing in the periodic array means that many different indices point to the same meaningful value. The index is computed from a combination of input-dependent registers, leading to multiple execution paths that converge to the same result. As a consequence, the real branch targets become difficult to identify, and symbolic analysis is forced to explore many feasible but nearly indistinguishable paths.

T2. State Explosion. Obfuscators can intentionally inflate the symbolic state space by inserting computations and loops that depend on input-dependent variables. These transformations create additional symbolic states or induce loops whose bounds depend on unknown input values, thereby multiplying the number of feasible execution paths. Such state forking cannot be eliminated by taint analysis or slicing without risking the removal of original program logic. The obfuscator may also apply opaque updates to internal data structures, creating implicit flows that further complicate simplification.

T3. Anti-Bruteforce. Obfuscators introduce artificial data dependencies between conditional branches and control flow updates to hinder brute-force branch exploration by flipping EFLAGS. If an attacker explores a branch by flipping EFLAGS without adjusting the corresponding data operands, the manipulated state causes the program to diverge into unintended regions. This forces analysis tools to satisfy both control flow and data-flow constraints, preventing naive brute-force exploration of conditional branches.

T4. Gadget Confusion. Obfuscators may treat normal data as gadget-like values or insert an address into an unaligned stack, breaking expected gadget boundaries. This obfuscation makes many positions in the chain look like possible gadget starts. As a result, pattern-matching analysis must examine numerous false candidates, leading to misleading gadget addresses and making it much harder to recover the actual gadget sequence.

T5. Indirect Branch. Obfuscators may rewrite structured switch-case logic into ROP-style indirect jumps whose targets depend on dynamically computed *RSP* updates and input-dependent jump-table indexing. This paradigm hides the original multi-branch control flow and prevents existing deobfuscation techniques from identifying all legitimate branch targets.

3.1.2. Instruction level

T6. Custom Stack-Operation Instructions. Obfuscators replace the native stack with a custom memory-based stack and rewrite simple operations such as *PUSH/POP* into address calculations. This transformation decouples the logical stack semantics from the physical stack pointer (e.g., *RSP*), making conventional stack tracking ineffective. Consequently, recovering the native stack effects requires analyzing complex arithmetic patterns rather than simply tracking stack pointers.

For example, the obfuscated code of the *PUSH R15* instruction transformed by Raindrop is shown in Fig. 2b. The obfuscated code allocates an intermediate stack array (Fig. 2b ①) to save the top of the original stack frame. The primary purpose is to replace the role of *ESP* in the regular program. When a stack operation is performed, the first operation is to obtain the size of the stack array (Fig. 2b ②) and then subtract the size from the stack array pointer to obtain the address of the top of the native stack (Fig. 2b ③), and then perform the stack operation (Fig. 2b ④). To execute the *PUSH R15* instruction, the native *RSP* is decremented by 8, and then the value of *R15* is moved to the target address.

T7. Redundant Instructions. Obfuscators frequently introduce dead instructions during ROP chain construction. While these instructions do not alter the program's core semantics, they significantly increase instruction noise. This pollution makes it harder for analysis tools to isolate meaningful operations and reconstruct the native semantics.

Redundant instructions manifest in three primary forms. First, for library-reuse-based ROP obfuscation, the gadgets are mainly extracted from various libraries during ROP chain generation. These gadgets inevitably contain dead code while forming the target semantics. Fig. 2c is the fragment of the ROP chain proposed by Yadegari et al. (2015). There are multiple instructions in a gadget, and some are dead code, as marked in red.

Second, obfuscators decompose a single instruction into equivalent sub-instruction sequences to adapt the gadgets extracted from the library. For example, the ROPfuscator will decompose the instruction *MOV DWORD PTR [ESP+4], 1* to the instruction sequence below.

```
POP EAX; RET; (MOV EAX, 0x4)
POP ECX; RET; (MOV ECX, ESP)
ADD EAX, ECX;
POP ECX; RET; (MOV ECX, 0x1)
MOV [EAX], ECX; RET;
```

Third, complex control flow obfuscation techniques (e.g., Threat T2) introduce semantically irrelevant instruction sequences. Unlike simple dead code, these instructions are often designed to appear semantically relevant to the program, making them difficult to eliminate through methods such as taint analysis.

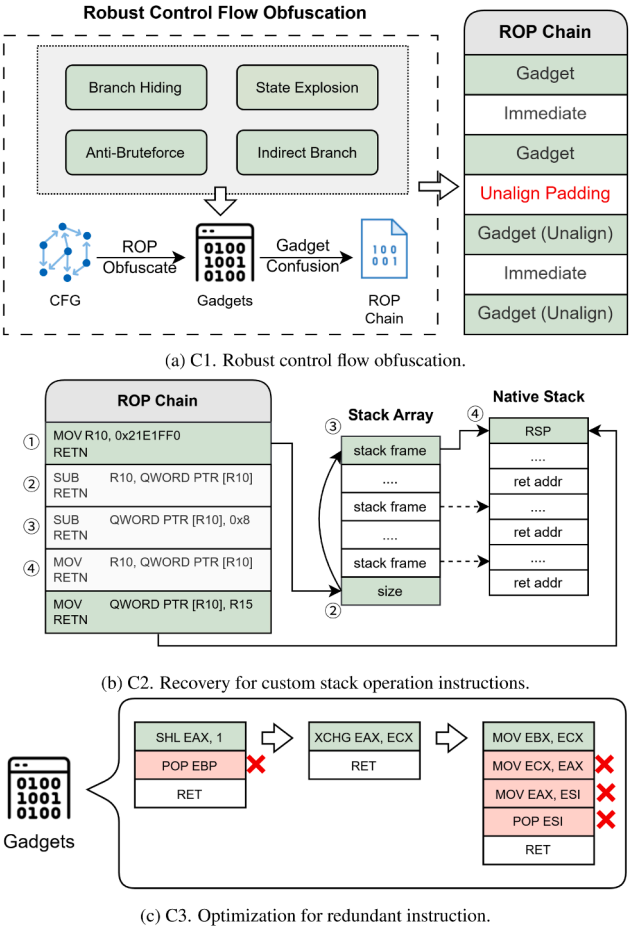
3.2. Challenge

In this section, we analyze the limitations of existing tools in conjunction with the threat model. We identify three key challenges impacting their effectiveness: *Robust Control Flow Recovery*, *Custom Stack Operation Instructions Recovery*, and *Redundant Instruction Optimization*.

C1. Robust Control Flow Recovery. As outlined in Threat T1–T5 and shown in Fig. 2a, modern obfuscators employ a synergistic combination of techniques Threat T1–T5 to thwart automated analysis. Static approaches (e.g., symbolic execution and taint analysis) are severely hindered by T1–T5, which trigger state explosion and lead to incomplete CFG reconstruction. Meanwhile, emulation-based dynamic methods are influenced by T3, thereby preventing brute-force exploration of conditional branches. In summary, existing methods are unable to recover the complete control flow when faced with such aggressive, multi-layered obfuscation.

C2. Custom Stack Operation Recovery. As explained in Threat T6 and shown in Fig. 2b, obfuscators replace the native stack mechanism with custom stack operations due to the ROP paradigm. Current ROP deobfuscation tools often struggle to interpret and convert these low-level operations into high-level stack operations. As a result, the deobfuscated code often contains confusing memory calculations, making it harder for reverse engineers to understand and analyze the program.

C3. Redundant Instruction Optimization. As discussed in Threat T7 and shown in Fig. 2c, redundant instructions inject significant semantic noise into the program. However, most existing deobfuscation approaches mainly focus on recovering control flow and pay little attention to removing such redundancy. As a result, the deobfuscated output often remains bloated and difficult to read, which complicates further manual or automated analysis.



(a) C1. Robust control flow obfuscation.

(b) C2. Recovery for custom stack operation instructions.

(c) C3. Optimization for redundant instruction.

Fig. 2. Example of ROP deobfuscation limitations.

3.3. Motivation

The challenges identified above expose a critical gap in current research: existing tools lack a holistic approach for robust control flow recovery, custom stack operation instructions recovery, and redundant instruction optimization.

Motivated by this, we designed `ROParser` with three core objectives: (1) to achieve robust control flow recovery against aggressive obfuscation (C1) by integrating Localized Symbolic Execution with Reduced Value Set Analysis; (2) to recover the semantics of custom stack operations (C2) using LLM-based few-shot learning; and (3) to ensure simplified output by eliminating redundant instructions (C3) through a specialized semantics-guided strategy.

By addressing these key challenges, `ROParser` more accurately recovers the original control flow of ROP-obfuscated programs and generates cleaner, more readable code, effectively reducing analysis costs.

4. Design

In this section, we present a novel methodology to address the challenges in ROP deobfuscation described above.

Overview. The `ROParser` workflow is organized as a three-stage pipeline, where each stage incrementally refines the recovered program representation.

First, *Robust Control Flow Recovery* reconstructs the complete CFG under aggressive obfuscation by combining *Localized Symbolic Execution*

with *Reduced Value-set Analysis*, producing a complete control flow for subsequent analysis.

Second, *Custom Stack Operation Instruction Recovery* operates on the recovered control flow and resolves custom stack operation instructions via backward slicing and LLM-based few-shot inference, restoring higher-level stack semantics.

Finally, *Semantics-Guided Redundant Code Elimination* simplifies the recovered instruction sequences by removing semantically irrelevant instructions and redundant basic blocks, yielding a concise and readable deobfuscated binary.

Together, these stages form a coordinated analysis pipeline that incrementally deobfuscates aggressive ROP-obfuscated binaries into an analyzable representation suitable for further reverse engineering.

4.1. Robust control flow recovery

Robust control flow recovery aims to reconstruct accurate successor relationships from aggressive ROP obfuscation, where traditional symbolic execution and pattern-based analysis fail.

Our approach integrates two complementary components-*Localized Symbolic Execution (LSE)* and *Reduced Value-Set Analysis (RVSA)*-to specifically counter the combined effects of control flow threat (T1-T5) mentioned in Section 3.1.1.

LSE focuses on the path-sensitive evolution of the ROP address pair ⟨RSP, RIP⟩, enabling precise gadget tracking and recovery of hidden branch targets. RVSA provides lightweight, constraint-guided reasoning to resolve critical comparisons, decode opaque constants, and efficiently enumerate feasible successors. Together, these techniques yield a resilient and scalable framework capable of recovering the native CFG even under aggressive and compound ROP obfuscation.

Algorithm 1 shows the workflow of the recovery pipeline. We formally model the recovery process as a state transition system over a state tuple (ℓ, Π, Δ), where:

- ℓ represents the current program location (⟨RSP, RIP⟩);
- Π represents the global constraints consisting of the set of *Explicit Constraints* collected by LSE;
- Δ is the symbolic expression store mapping registers and memory.

Algorithm 1 maintains a worklist $\mathcal{W}$ of state tuples corresponding to gadget targets to be analyzed. For each state, LSE and RVSA are cooperatively applied to enumerate all legal successor states, which are then added back to $\mathcal{W}$ for further exploration. This process iterates until $\mathcal{W}$ becomes empty, at which point the recovered CFG is complete.

4.1.1. Localized symbolic execution

To recover control flow under aggressive ROP obfuscation, we introduce LSE, a targeted symbolic analysis that focuses exclusively on control flow-relevant semantics.

The LSE design follows a two-stage execution strategy that balances efficiency and precision. The first stage performs a fast, repetition-limited, bounded exploration to approximate the overall control flow structure. The second stage is invoked only when necessary to resolve uncertain successors through dependency-guided symbolic recovery. This separation allows LSE to recover accurate control flow successors while avoiding the prohibitive cost of complete symbolic execution.

Stage 1: Bounded Symbolic Exploration. In the first stage (blue part in Algorithm 1), LSE performs a bounded symbolic execution with the state tuple (ℓ, Π, Δ). Each basic block is explored at most once, preventing repeated branch exploration and unbounded symbolic growth. Successor states $\mathcal{B}$ (including conditional branches) are obtained through symbolic execution. Subsequently, each successor state $(\ell_{next}, \Pi, \Delta') \in \mathcal{B}$ with a concrete and valid RSP_{next} is added to $\mathcal{S}$, after which V, E, and $\mathcal{W}$ are updated accordingly.

In practice, this strategy is sufficient to recover the complete control flow in most cases. However, two exceptional situations may arise when computing successor states:

Algorithm 1: Robust Control Flow Recovery via LSE and RVSA.

Input: Entry location ℓ_{entry}, ROP Chain $\mathcal{R}$
Data : Worklist $\mathcal{W}$, CFG $G = (V, E)$, Symbolic Store Δ, ProcessedSet $\mathcal{V}_{done}$

$\mathcal{W} \leftarrow \{(\ell_{entry}, \text{true}, \emptyset, \Delta_{init})\}$;
$V \leftarrow \{\ell_{entry}\}, E \leftarrow \emptyset, \mathcal{V}_{done} \leftarrow \emptyset$;
while $\mathcal{W} \neq \emptyset$ **do**
  $(\ell, \Pi, \Delta) \leftarrow \text{pickNext}(\mathcal{W})$;
  // LSE Stage 1
  if $\ell \in \mathcal{V}_{done}$ **then**
    continue
  $\mathcal{V}_{done} \leftarrow \mathcal{V}_{done} \cup \{\ell\}$;
  $\mathcal{B} \leftarrow \text{BoundedSymExec}(\ell, \Pi, \Delta)$;
  $\mathcal{S} \leftarrow \emptyset$;
  foreach $(\ell_{next}, \Pi', \Delta') \in \mathcal{B}$ **do**
    $RSP_{next} \leftarrow \text{eval}(\Delta', RSP)$;
    if $isConcrete(RSP_{next}) \wedge isValid(RSP_{next})$ **then**
      // Normal Successor
      $\mathcal{S} \leftarrow \mathcal{S} \cup \{(\ell_{next}, \Pi', \Delta')\}$;
    else if $isSymbolic(RSP_{next})$ **then**
      // RVSA:Symbolic Successor
      $\mathcal{V}_{cand} \leftarrow \text{RVSA}(\Delta', \Pi')$;
      foreach $val \in \mathcal{V}_{cand}$ **do**
        $\Delta_{new} \leftarrow \Delta' \cup \{RSP \leftarrow val\}$;
        $\Pi' \leftarrow \Pi' \wedge (RSP = val)$;
        $\mathcal{S} \leftarrow \mathcal{S} \cup \{(\text{target}(val), \Pi', \Delta_{new})\}$;
    else // LSE Stage 2:Invalid Successor
      $Slice \leftarrow \text{BackwardSlice}(RSP_{next}, \mathcal{R})$;
      $I_{irr}, I_{rel} \leftarrow \text{SemanticPrune}(Slice, \mathcal{R})$;
      $(\Delta_{rec}, \ell_{rec}) \leftarrow \text{PathSensReExec}(I_{irr}, I_{rel}, \mathcal{R})$;
      $\mathcal{S} \leftarrow \mathcal{S} \cup \{(\ell_{rec}, \Pi', \Delta_{rec})\}$;
  foreach $(\ell', \Pi', \Delta') \in \mathcal{S}$ **do**
    if $\ell' \notin V$ ***or*** $isUnvisitedEdge(\ell, \ell')$ **then**
      $V \leftarrow V \cup \{\ell'\}$;
      $E \leftarrow E \cup \{(\ell, \ell')\}$;
      $\mathcal{W} \leftarrow \mathcal{W} \cup \{(\ell', \Pi', \Delta')\}$;
return $G(V, E)$

- *Symbolic* RSP_{next}, indicating polymorphic or input-dependent targets, which are deferred to RVSA for lightweight resolution.
- *Invalid* RSP_{next}, suggesting strict path constraints beyond the scope of bounded exploration. This specific failure will trigger Stage 2.

Stage 2: Dependency-Guided Symbolic Recovery. In the second stage (gold part in Algorithm 1), LSE performs a dependency-guided symbolic recovery to recover the invalid *RSP* successors while preserving control flow precision and avoiding state explosion (Threat T2).

First, LSE performs a *Dependency Analysis* by *Backward Slicing* starting from the *RSP* update instruction. The slice conservatively tracks all operations that directly or indirectly affect *RSP*, including registers and memory locations that influence conditional execution, ensuring that all control-flow-critical dependencies are retained, yielding the slicing result *Slice*.

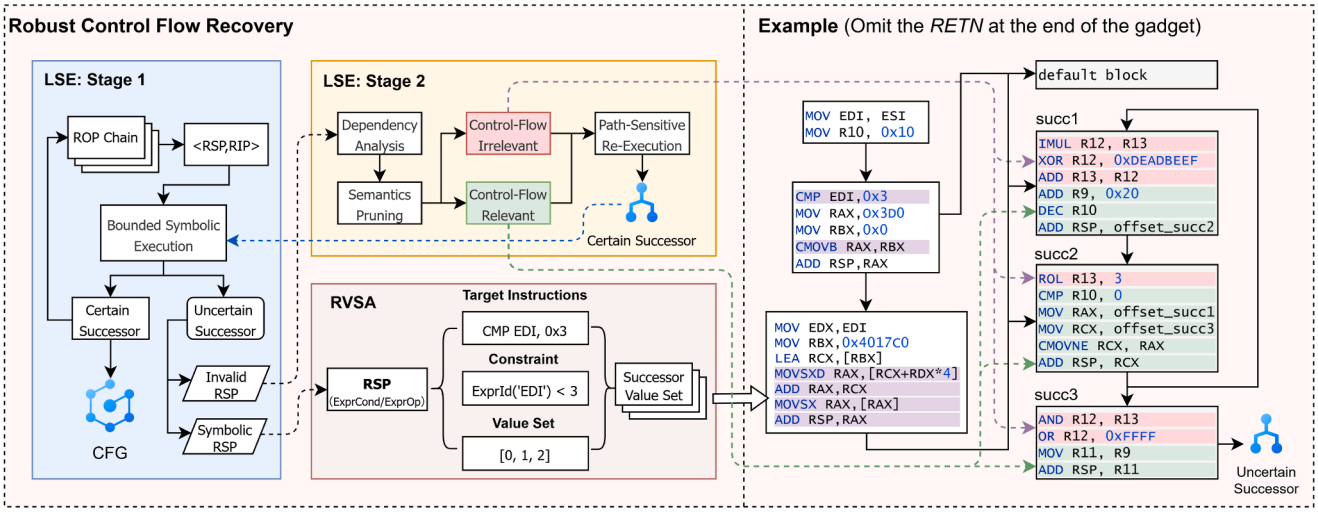


Fig. 3. Overview and case study of robust control flow recovery.

Second, LSE performs a *Semantics Pruning*, partitioning the instructions into two categories:

- *RSP-relevant* I_{rel}, which directly or indirectly affect *RSP*.
- *RSP-irrelevant* I_{irr}, which do not contribute to any RSP update or branch predicate.

LSE proceeds with a *Path-Sensitive Re-Execution* from the program entry, strictly following the CFG. I_{rel} are analyzed through complete symbolic execution to compute the successor $\langle RSP, RIP\rangle$ accurately. I_{irr} instructions are skipped during symbolic execution; however, control-flow transfer instructions (e.g., `RETN`) are preserved and executed to maintain the continuity of the ROP chain. Since backward slicing has verified that these instructions do not affect *RSP* or branch decisions, this concretization does not compromise control flow correctness.

By restricting complete symbolic reasoning to control-flow-critical instructions and aggressively pruning irrelevant symbolic states, Stage 2 recovers the valid successor $(\Delta_{rec}, \ell_{rec})$ efficiently, without reintroducing the state explosion (Threat T2) avoided in Stage 1. Notably, by leveraging symbolic execution, our approach naturally provides a precise solution for unaligned gadget addresses, directly solving Threat T4. Together, the two stages of LSE provide a robust and practical mechanism for recovering control flow in heavily ROP-obfuscated binaries.

4.1.2. Reduced value-set analysis

While LSE efficiently reconstructs the main control flow, certain branches remain unresolved due to input-dependent or polymorphic *RSP* updates. To handle these cases, we introduce *RVSA*, a lightweight method that complements LSE by resolving ambiguous successors without incurring the high cost of full symbolic exploration.

As shown in the pink section of Algorithm 1, RVSA is seamlessly integrated into the LSE workflow. It operates by leveraging explicit constraints Π' accumulated during the LSE exploration from control-flow-critical instructions, such as `TEST` and `CMP`. These constraints are leveraged to perform a targeted value-set analysis, resolving symbolic branch targets into a concrete successor set $\mathcal{V}_{cand}$. For every resolved value $val \in \mathcal{V}_{cand}$, RVSA derives the target address pair $\langle RSP, RIP\rangle$ via the function target(val). It then refines the global path predicate Π by binding the RSP to the resolved value, adds the resulting state to the successor set $\mathcal{S}$, and propagates it to the worklist $\mathcal{W}$.

The specific implementation of RVSA relies on constructing a preliminary candidate set, denoted as $\mathcal{V}'_{cand}$, for the uncertain symbols within the symbolic *RSP* expression, guided by the collected constraints Π. The recovery scheme employs the following three key strategies:

Opaque-Encoded Branch Recovery. To address Threat T1 (hiding branches), RVSA evaluates the resulting candidate values for the *RSP* symbol expression using the candidate set $\mathcal{V}'_{cand}$. If all computations yield the same constant, the expression is classified as *opaque-invariant*, indicating that the *RSP* effectively converges to a deterministic successor.

This approach successfully recovers encoded branch targets without requiring prior knowledge of the specific obfuscation scheme, while actively avoiding unnecessary symbolic state explosion (Threat T2).

Effective Branch Exploration. To address Threat T3 (anti-bruteforce branches), RVSA concretizes the symbolic expression using the candidate set $\mathcal{V}'_{cand}$. Anti-bruteforce patterns demand that the symbols within the *RSP* symbol expression strictly align with branch constraints to compute the correct target. By concretizing the symbolic expression using the candidate set $\mathcal{V}'_{cand}$, we ensure that these rigorous conditions are satisfied, enabling the precise recovery of the successor value.

This approach recovers the correct branch successor without performing full symbolic exploration, while ensuring that only valid ROP-chain-progressing paths are considered.

Indirect Branches Exploration. To address Threat T5-resolving branches rewritten as RSP-indexed indirect jumps (e.g., switch-case via jump tables)-RVSA iterates over each candidate value Δ_i within the candidate set $\mathcal{V}'_{cand}$. By substituting these candidates into the uncertain symbols of the *RSP* expression, we derive potential successor states:

$$RSP_{\text{next},i} = RSP_{\text{cur}} + \Delta_i.$$

Only syntactically valid ROP addresses are registered as legitimate successors. It allows RVSA to efficiently reconstruct complete indirect multi-branch structures, even under symbolic indexing or jump-table diversification.

Summary. By integrating these strategies, RVSA reliably recovers branch targets that remain unresolvable by LSE alone. It serves as a critical complement to the two-stage LSE mechanism, ensuring comprehensive CFG reconstruction even under aggressive ROP obfuscation.

4.1.3. Case study: Workflow analysis

To demonstrate the coordinated workflow of LSE and RVSA, we present a case study involving a gadget chain with switch-case and loop structure, as shown in Fig. 3. The switch-case structure implements indirect jumps to three successors and a default block through the jump table. Specifically, *succ3* targets an uncertain successor that depends on computations performed in the preceding branches. This structure relies on strict sequential execution (loop of succ1 and succ2) to determine

the target, making it a clear example to demonstrate the two-stage LSE analysis.

Phase 1: LSE Stage 1. The analysis begins with LSE Stage 1, which performs *Bounded Symbolic Exploration* from the gadget entry. It locates ⟨*RSP*, *RIP*⟩ pairs and resolves most control flow branches via symbolic execution, exploring each basic block at most once. During this phase, two specific scenarios trigger the involvement of auxiliary modules:

- *Symbolic RSP Successor:* For the dispatcher block, the analysis encounters a successor where the RSP evaluates to a symbolic value (indicated in purple in Fig. 3). This implies a multi-branch structure dependent on the initial symbolic value of *ESI*. Consequently, the analysis transitions to the RVSA module to resolve the precise value set.
- *Invalid RSP Successor:* For *succ3*, the analysis calculates the successor target (e.g., via `ADD RSP, R9`) but yields an invalid address. This occurs because the bounded exploration of Stage 1 did not capture the necessary computations from the preceding blocks. This anomaly triggers the transition to the LSE Stage 2.

Phase 2: RVSA. The RVSA module continuously collects explicit constraints (e.g., `CMP EDI, 0x3; CMOVB;`) active in the execution context. Upon receiving the symbolic RSP result, it uses these accumulated constraints to solve for the finite set of feasible values for the dependent symbol (i.e., `ESI` $\in \{0, 1, 2\}$). By substituting these values into the symbol, RVSA resolves the valid set of RSP targets, thereby recovering all accurate successors for the multi-branch structure.

Phase 3: LSE Stage 2. To resolve the invalid successor in the succ3, LSE employs *Dependency-Guided Symbolic Recovery*. It performs a *Backward Slicing* starting from the target instruction (e.g., `ADD RSP, R9` in *succ3*). By combining conservative alias analysis for registers and memory, the analyzer partitions the instruction sequence into *control flow Irrelevant* (pink) and *control flow Relevant* (green) sets.

By explicitly skipping the computationally expensive irrelevant instructions (e.g., complex arithmetic in pink blocks), LSE performs *Path-Sensitive Re-Execution* from the entry using the identified sequence. By strictly following the program's control flow for relevant instructions, LSE correctly executes the modification instructions for *R9* in the loop of *succ1* and *succ2*. It ultimately yields the correct successor RSP address for the *succ3*, completing the CFG recovery.

This example demonstrates how our method operates in practice. Combined with the various RVSA strategies described in Section 4.1.2, our approach can analyze ROP payloads in complex scenarios and achieve complete control flow recovery against aggressive obfuscation.

4.2. Custom stack operation instruction recovery

After the control flow recovery process, the instructions that require further recovery due to the impact of Challenge C2 can be categorized into two types.

The first category comprises control flow instructions, primarily *CALL*, *RET*, *JMP*, and *JCC* instructions, which can be accurately identified and restored during the control flow recovery process. For *CALL* and *RET*, these instructions interact with the custom stack to maintain new stack frames and implement function return. The instruction *CALL* is identified when the successor target of the current instruction is not a valid gadget address in the ROP chain. The instruction *RET* is identified when the custom stack is emptied, and the instruction retrieves the return address from the stack frame as its successor target. Jump instructions like *JMP* and *JCC* implement jumps by modifying the stack pointer *RSP*. These instructions can be identified by the *RSP* register modification behaviors at the end of basic blocks. These instructions can be accurately identified during the control flow recovery process without requiring additional processing.

The second category comprises direct stack operation instructions, such as *PUSH* and *POP*, which interact directly with the user stack. As shown in Fig. 2b, these instructions are obfuscated to include an intermediate layer that addresses from a stack array before manipulating the native stack. This transformation converts simple semantic information into complex semantics, complicating the deobfuscation process when attempting to recover the original semantics. Due to factors such as opcode diversity, varied operand types, and multiple addressing modes, these instructions exhibit numerous behavioral patterns, making it difficult to recover them to the deobfuscated form through simple pattern matching. Consequently, accurately recovering these instructions requires a dedicated approach to ensure both correctness and scalability.

Prompt: Custom Stack Operation Instruction Recovery

Assembly Deobfuscation Expert

Task: Convert ROP-obfuscated stack instructions to standard x86 instructions
Core Rules:
1.Obfuscated multiple instructions → Output deobfuscated instructions
2.Transform to RSP and delete intermediate registers
3.Preserve immediate values in original format
4.Output non-obfuscated instructions unchanged
5.Unrecognized patterns → 'UNKNOWN'
6.Output the results directly without any explanation
Obfuscation Patterns:
- Starts with fixed address
- Multiple memory dereference
- Stack operation semantics
Examples:
few-shot learning examples.
I/O format:
Input:
"'User instructions"'
Output:
"'Deobfuscated instructions"'

Fig. 4. Excerpts of the LLM prompt used for custom stack operation instruction recovery.

To address this challenge, we propose a novel approach that leverages the robust contextual semantic understanding and logical reasoning capabilities of Large Language Models (LLMs). Our solution employs a few-shot learning paradigm (Huang et al., 2023) to accurately recover custom stack operation instructions. This method consists of two stages:

Stage 1. Code Slicing. This stage focuses on code slicing through static analysis to extract all custom stack operation instruction sequences for subsequent analysis. As shown in Fig. 2b, the obfuscated program reorganizes the user stack in memory so that each instruction sequence begins with an access operation to the stack array address. This characteristic enables precise identification of instruction starting points.

From these starting points, we perform a forward analysis based on DEF-USE chain analysis. First, we generalize the DEF-USE relationship to δ-correlated instruction sequences as follows:

Let $I = \{i_1, i_2, \ldots, i_n\}$ be an instruction sequence. For a given correlation window size $\delta \geq 1$, we define the δ-correlated instruction sequence as follow:

$$I_c = \left\{ i_j \in I \mid \exists m \in [j-\delta, j-1], DEF(i_m) \cap USE(i_j) \neq \emptyset \right\}$$

The variable δ represents the maximum allowed distance between correlated instructions. The correlation window size δ can be dynamically adjusted to accommodate obfuscated instruction sequences under different obfuscation conditions. For the specific case of custom stack operation instruction recovery, we establish a strict immediate dependency by setting $\delta = 1$. This configuration enforces that each instruction must maintain a direct DEF-USE relationship with its subsequent instruction, formally expressed as:

$$\forall i_j \in I_c,\ DEF(i_j) \cap USE(i_{j+1}) \neq \emptyset$$

Use this algorithm to obtain the correlated instruction sequence starting from each instruction starting point for subsequent instruction recovery. Since non-obfuscated instructions may also be included in the analysis sequence under this method, we introduce an instruction sequence length threshold to prevent uncontrolled sequence expansion. The problem of distinguishing obfuscated instructions is addressed by the LLM in stage 2.

Stage 2. Instruction Deobfuscation. This stage employs a few-shot prompting approach to deobfuscate the complex semantics of custom stack operation instructions from the instruction sequences obtained in the previous state. As shown in Fig. 4, we configure the large language model (LLM) as a binary deobfuscation expert, specifying the primary deobfuscation tasks and obfuscation patterns. Through experimentation, we found that providing 5–6 well-designed few-shot learning examples can achieve high deobfuscation accuracy.

To address efficiency problems caused by LLM inference time and network latency, we implemented a parallel processing strategy. Rather than processing instructions sequentially, we group them by function and perform batch analysis, significantly reducing overall processing time. Moreover, if the output during LLM instruction deobfuscation does not match the expected instruction format, the deobfuscation process is reapplied to the target instruction to ensure the correctness of the results.

This approach effectively leverages the LLM's capabilities. The experimental results demonstrating the effectiveness of this method will be presented in Section 6.

4.3. Semantics-guided redundant code elimination

To address the challenge of redundant instruction optimization (C3), we propose a multi-granularity optimization strategy that targets redundancy at both the instruction and basic-block levels. This approach applies instruction-level constant propagation and liveness analysis to simplify instruction sequences, while employing semantics-driven block pruning to eliminate redundant control flow blocks.

4.3.1. Instruction-level elimination

To reduce semantically irrelevant instructions generated during ROP gadget extraction, we perform instruction-level redundancy elimination using two complementary analyses: partial forward constant propagation and liveness analysis.

Constant Propagation. To further simplify instructions decomposed by obfuscators, we apply a partial forward constant propagation at the instruction level as detailed in Algorithm 2. Unlike standard constant propagation, which often considers only literal immediates, we extend the definition of constants to include: immediate values, registers, and simple expressions combining a register with an immediate (e.g., sum or difference). This broader view allows us to propagate more semantics while preserving correctness.

The propagation operates on the basic-block level and maintains a worklist of active constants, where each entry tracks a target register, its propagated value, and the originating instruction. During traversal, each instruction's operands are updated according to the current worklist, ensuring that subsequent instructions can leverage previously propagated constants. After updating an instruction, any superseded constants in the worklist are deactivated to reflect new assignments.

We enforce syntactic correctness by validating each updated instruction before committing changes to the CFG. Instructions that cannot be safely rewritten are left unchanged, preventing invalid assembly instructions. Formally, for an instruction I, a propagatable operand arg is replaced with its corresponding constant value c from the worklist if doing so maintains instruction validity.

This approach enables constant propagation iteratively, simplifying instruction sequences without introducing invalid operations. Combined with instruction-level liveness analysis, it further removes redundant instructions, reducing the overall complexity of recovered ROP gadgets.

Algorithm 2: Constant propagation algorithm.

```
Require: Control Flow Graph G = (V, E)
foreach B ∈ V do
    W ← ∅ foreach I ∈ B do
        args ← I.getArgs() ;
        foreach arg ∈ args do
            foreach (OUT'_R, OUT'_I, I') ∈ W do
                if arg = OUT'_R then
                    I'' ← I.update_arg(OUT'_I) ;
                    if check_Inst(I'') = true then
                        B.update_inst(I, I'') ;
                        B.delete_inst(I')
        (OUT_R, OUT_I) ← F_I(I)
        foreach (OUT'_R, OUT'_I, I') ∈ W do
            if OUT_R = OUT'_R then
                W.delete((OUT'_R, OUT'_I, I')) ;
        if OUT_I is propagatable then
            W ← W ∪ {(OUT_R, OUT_I, I)} ;
```

Algorithm 3: Dead code identification.

```
Require: Control Flow Graph G = (V, E)
Ensure : DeadCodeList: List of removable instructions
Initialize: DeadCodeList ← ∅ ;
foreach B ∈ V do
    foreach I ∈ B do
        if ¬IS_SIDE_EFFECTING(I) then
            if DEF[I] ∩ OUT[I] = ∅ then
                DeadCodeList ← DeadCodeList ∪ {I}
```

Liveness Analysis. To eliminate redundant code inherited from library gadgets and instruction decomposition, we employ a fine-grained liveness analysis. Unlike standard approaches that operate at the basic block level, we adapt the backward data-flow analysis to compute liveness sets (IN[I] and OUT[I]) for individual instructions. This granularity allows us to pinpoint and remove useless instructions within the unstructured fragments of ROP gadgets.

The core of our optimization lies in a specialized retention policy based on *Semantic Side-Effects*. In the context of ROP execution, we define an instruction I as side-effecting if it performs critical operations that must be preserved regardless of register liveness. Specifically, side-effecting instructions include:

- *Control Flow:* Instructions altering the execution path (e.g., `CALL`, `JMP`, `JCC`, `RET`, `HLT`).

- *Memory/Stack:* Memory writes or explicit Stack Pointer updates (e.g., `MOV [RAX], RBX`, `ADD RSP, 0x8`).
- *EFLAGS:* Instructions modifying `EFLAGS` (e.g., `CMP`, `TEST`) if subsequently used by conditional jumps.
- *Return Values:* Definitions of return registers (e.g., `RAX`) at function exits.

Based on this definition, we identify dead code by checking both the instruction's inherent side effects and the liveness of its defined variables. As detailed in Algorithm 3, an instruction is marked as redundant if it is not side-effecting and none of its defined registers are live immediately after execution.

4.3.2. Basic-block-level elimination

Beyond individual instructions, aggressive obfuscation configurations often inject redundant basic blocks of complex logic, leading to state explosion (Threat T2). These blocks perform intensive, input-dependent computations to confuse analysis, yet they often have little global semantic impact.

To eliminate such redundancy, we propose a block-level elimination algorithm targeting single-successor basic blocks (i.e., linear execution paths). We define a block B as *Semantically Irrelevant* if it satisfies two formal conditions:

- *Absence of Side-Effects:* The block must not execute operations that alter the global system state (e.g., memory writes, stack pointer updates, or system calls). Formally:

$$\forall I \in B, \neg \text{IS\_SIDE\_EFFECTING}(I) \tag{1}$$

- *Transient Definitions:* The values computed within the block must not be consumed by any subsequent block. Let $\text{DEF}[B]$ be the set of registers (including `EFLAGS`) modified in B, and $\text{OUT}[B]$ be the set of live variables at the block's exit. The condition is:

$$\text{DEF}[B] \cap \text{OUT}[B] = \emptyset \tag{2}$$

This empty intersection guarantees that all calculations inside B are dead-they are either overwritten locally or never read by the program's future execution.

For any single-successor block B satisfying these criteria, we perform *Edge Retargeting*: we redirect all incoming edges from B's predecessors directly to B's unique successor, effectively removing the redundant logic from the CFG.

5. Implementation

We implemented a prototype of `ROParser` using approximately 3000 lines of Python code. We used `miasm` (Desclaux, 2012), a lightweight symbolic execution and binary analysis framework, for symbolic analysis and value set analysis. For custom stack operation instruction recovery, we integrated the DeepSeek_V3 model (Liu et al., 2024) into the system for instruction analysis. For compiler optimization in dead code elimination, we used the method *regs_access* of disassembler *Capstone* to obtain the DEF and USE sets of the instructions. Some instructions required manual assignment, such as the CALL instructions for the X64 architecture. Therefore, we manually added the commonly used parameter registers (*RDI, RSI, RCX, RDX, R8, R9*) to the USE collection. To support discontinuous ROP chains in the ROPfuscator, we analyzed micro-ROP chains and added their results to the overall control flow.

6. Evaluation

In this section, we evaluate the deobfuscation ability of `ROParser`. We focus our evaluation on the following research questions.

- **RQ1: Effectiveness.** What is the deobfuscation capability for existing ROP obfuscation methods? (Section 6.2)

Table 1
Effectiveness evaluation result of `ROParser`.

Metric	D_1	D_2	D_3
Programs	101	101	68
Functions	228	228	68
Blocks	1486	1486	68
Edges	1766	1766	N/A
Gadgets	30,283	4178	1523
Instructions	7631	7631	1523
CFG Similarity	97.97%	96.13%	N/A
Obfuscate ratio	99.16% (2719/2742)	N/A N/A	N/A N/A
Eliminate ratio	N/A N/A	25.56% (1068/4178)	15.36% (234/1523)

Table 2
Robustness of `ROParser` under enhanced obfuscation configurations.

Obfuscation Config	CFG (%)	CFG-opt (%)	Time (s)
D_1-Basic	97.97	97.97	2.52
D_1-Gadget Confusion	97.97	97.97	2.69
D_1-Anti-Bruteforce	97.97	97.97	2.85
D_1-Anti-ROP-Disasm	57.78	92.40	4.94
D_1-State Explosion	38.26	93.32	8.39
D_1-Extreme	27.21	89.16	8.65

- **RQ2: Performance.** How does the deobfuscation process perform in terms of time efficiency and resource expenditure? (Section 6.3)
- **RQ3: Comparison.** How does `ROParser` compare to the state-of-the-art in terms of effectiveness and performance? (Section 6.4)
- **RQ4: Practicality.** Does the design of this method exhibit strong practicality in the real world? (Section 6.5)
- **RQ5: Robustness.** Whether the LLM process is robust to inference variability and different LLM choices, and how its accuracy scales with model capacity. (Section 6.6)

6.1. Experimental setup

We conducted our evaluation on Ubuntu 22.04 on a 13th Gen Intel(R) Core(TM)i7-13800H with 32GB RAM.

Benchmark: For our experimental evaluation, we selected the obfuscation-benchmarks repository on GitHub (TUM-i4, 2015) as our benchmark. This collection of C programs, developed by researchers at the Technical University of Munich (TUM), serves as a standardized test set for systematically evaluating the resilience of various obfuscation techniques against both manual and automated reverse-engineering attacks. The benchmark includes basic algorithms such as factorial, sort, search, GCD, and LCM, as well as programs with specific control flow complexity. The diversity and scale of these programs provide a robust foundation for evaluating the effectiveness of ROP deobfuscation methods.

Dataset: Four distinct datasets, incorporating both real-world ROP chains and synthetic benchmarks, support our evaluation. To generate the obfuscated benchmarks, we employ two state-of-the-art ROP obfuscators-Raindrop and ROPfuscator-which represent the current SOTA in code obfuscation research. These obfuscators were selected for their advanced capabilities to generate complex, real-world-like ROP chains and for their widespread recognition in the literature. The obfuscated programs feature intricate control flow structures, offering a rigorous evaluation for control flow recovery. ROPOB is another approach related to ROP obfuscation; however, as it is not publicly available, it was not included in our evaluation. By incorporating these highly advanced obfuscators, we ensure a comprehensive assessment that reflects the challenges posed by contemporary ROP obfuscation techniques.

The first dataset D_1 is generated using the obfuscation method Raindrop. Since Raindrop operates at the binary level, we compile the programs into 64-bit binaries on Ubuntu 20.04 using GCC, with PIE disabled via the -no-pie flag.

Raindrop exposes several orthogonal obfuscation mechanisms that significantly increase the difficulty of recovering control flow and ROP semantics. To demonstrate the effectiveness of our method against the adversarial techniques in the threat model, we therefore construct six variants of D_1, each targeting a distinct source of complexity:

D_1-Basic: The default setting used in the original evaluation.

D_1-Anti-ROP-Disasm: Enables a single -o flag, which injects opaque predicates to hinder static disassembly and disrupt linear sweep disassemblers.

D_1-State Explosion: Enables two -o flags (-o -o), leading to severe symbolic state explosion, a known challenge for symbolic execution engines.

D_1-Gadget Confusion: Enables the –confusion flag, introducing arithmetic obfuscation on gadget parameters and intermediate states, substantially increasing the difficulty of accurate semantic recovery.

D_1-Anti-Bruteforce: Enables the –anti-bruteforce flag, injecting branches that intentionally defeat brute-force branch resolution and basic heuristics.

D_1-Extreme: A stress-test configuration combining all aggressive options.

These transformations capture different dimensions of ROP-oriented obfuscation difficulty, enabling a more realistic and comprehensive robustness evaluation.

The second dataset D_2 is generated using the obfuscation method ROPfuscator. These programs were compiled from the benchmark source code into 32-bit binaries on Ubuntu 18.04. Unlike Raindrop, ROPfuscator does not generate ROP gadgets internally; instead, it retrieves them from external libraries. We configure ROPfuscator to extract gadgets from *libc.so.6*. To focus solely on evaluating ROP deobfuscation, we disable other obfuscation features such as opaque predicates and instruction hiding in ROPfuscator.

The third dataset D_3 consists of real-world ROP chains extracted from the exploit code of 68 distinct CVEs. The CVE ROP chains were collected from publicly available repositories (Rapid7, 2011; Google, 2024), and analyzed using ROParser to evaluate usability.

The fourth dataset D_4 also includes a representative selection of obfuscated binaries from GNU Coreutils (`test`, `whoami`, `date`, `dirname`, `cat`, `mkdir`, `ls`, `base64`), which were obfuscated using Raindrop with an aggressive configuration. Detailed program data is recorded separately in a Table 5.

The first six rows of Table 1 present the basic information of three datasets, including the number of programs, functions, basic blocks, edges, instructions, and gadgets in the ROP chains. Each dataset is sufficiently large to support the comprehensive evaluation of `ROParser`. Among them, the CVE ROP chains in dataset D_3 are constructed from real CVE exploitation code, so their control flow is relatively simple, consisting of only one basic block. Still, the complexity of gadgets can support the evaluation of genuine ROP chains.

Baseline Technique: We compare `ROParser` against the state-of-the-art ROP deobfuscation tool, ROPdissector. ROPMEMU and ROPdissector employ dynamic analysis methods, so we selected the latest ROPdissector as one of the baseline tools. We excluded DeRop (Lu et al., 2011) and Yadegari et al. (2015) because they lack open-source implementations. Similarly, Syntia (Blazytko et al., 2017) was not considered as a baseline, as it targets trace-based semantic synthesis and does not support complete ROP deobfuscation. Since the original ROPdissector supports only the PE binary format, we modified the source code to support the ELF format and made a few adjustments to fit the datasets.

Ground Truth: For the datasets D_1 and D_2, since the programs are obfuscated at both the binary and source levels, we used the CFG of the original program compiled by GCC as the basis. For the dataset D_3, we selected the manually analyzed results for the original control flow and used them as a basis after removing all dead code via standard live variable analysis.

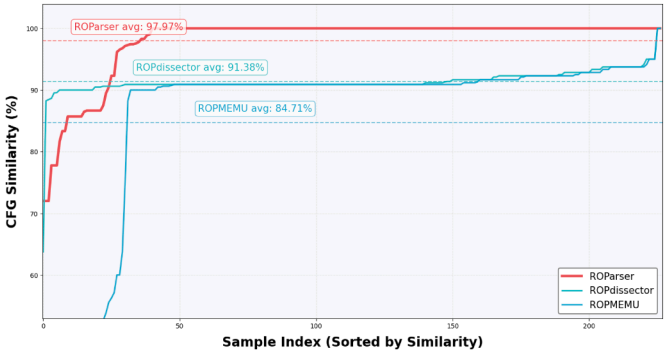


Fig. 5. Evaluation result of CFG similarity.

Measurement Metrics: To evaluate the deobfuscation performance of `ROParser`, we use two metrics: CFG similarity and semantic similarity. We quantitatively measure the CFG similarity using the algorithm described by Yadegari et al. (2015). First, the edit distance $\delta(G1, G2)$ between two CFGs is computed using the function implemented in the Python library *Networkx*. Then the similarity equation is shown in Eq. (3). The absolute value of CFG is the total number of basic blocks and edges.

$$simCFG(G_1, G_2) = 1 - \frac{\delta(G_1, G_2)}{|G_1| + |G_2|} \tag{3}$$

To measure semantic similarity, we employ qualitative research based on runtime behavior equivalence, where a deobfuscated program is considered correct if it exhibits identical semantic behavior to the original program. Since our deobfuscation method is based on the function level, we add the corresponding program data segment mapping at runtime and directly return the expected results for called subfunctions and system calls, ensuring the program executes normally. Since there is no parameter initialization process, the program parameters *argc* and *argv* are passed directly by dynamically modifying the memory.

To evaluate the optimization strategy, we use the ratio of custom stack operation instruction deobfuscation to assess the method described in Section 4.2. The ratio is calculated by dividing the deobfuscated number by the total number of custom stack operation instructions.

We use the ratio of redundant elimination to measure the effectiveness of the optimization method stated in Section 4.3. The ratio is calculated by dividing the number of eliminated instructions by the total number of instructions.

6.2. Effectiveness (RQ1)

We evaluated the core functionalities of `ROParser` for ROP deobfuscation in the effectiveness evaluation. We assess several key performance metrics during the deobfuscation process, including: control flow deobfuscation, custom stack operation instruction recovery, and redundant instruction elimination.

Control Flow Deobfuscation. The seventh row of Table 1 presents the control flow recovery performance of `ROParser`. Since dataset D_3 does not contain complex control flow structures, it is excluded from this evaluation. For dataset D_1-Basic, the average CFG similarity is 97.97%, indicating that `ROParser` recovers most of the control flow. The minor inaccuracies observed are mainly caused by the redundant splitting of the same basic block and the redundant edges originating from the terminal blocks ending with the `RETN` instruction. These redundant control flows do not affect the core control flow information.

For dataset D_2, the average CFG similarity is 96.13%. Since ROPfuscator only rewrites local control flow transitions to gadgets during ROP obfuscation, many complex branches, such as switch cases, retain the original program instructions.

Therefore, reconstructing the ROP-obfuscated direct control flow transformations is sufficient to recover the overall CFG. The minor inaccuracies caused by redundant splitting of the basic blocks during recovery do not affect the core control flow information.

To further evaluate the robustness of our control flow recovery method, we evaluate `ROParser` under six variants of dataset D_1. As shown in Table 2, the column CFG reports the raw CFG similarity immediately after control flow recovery. In contrast, CFG-opt shows the similarity after applying post-processing optimization that incorporates lightweight pattern matching to eliminate redundant blocks due to state explosion and redundant branches with opaque predicates.

Across all configurations, `ROParser` demonstrates strong resilience against the control flow threats described in our threat model. For transformations such as Gadget Confusion and Anti-Bruteforce, the raw CFG similarity already reaches 97.97%, matching the Basic configuration. It is because these transformations do not introduce structural redundancy, and symbolic execution naturally normalizes the arithmetic variations or resolves the implicit branch semantics.

More challenging configurations explicitly designed to break symbolic execution-such as Anti-ROP-Disasm and State Explosion-do cause significant degradation in the initial CFG similarity (57.78% and 38.26%, respectively). These cases introduce opaque predicates and a large number of misleading conditional branches, which inflate the CFG with non-semantic blocks and induce state explosion during symbolic execution. However, once we remove redundant branches and optimize semantically irrelevant block sequences, the CFG similarity increases to 92.40% and 93.32%. This substantial improvement shows that the majority of the structural divergence originates from redundant control flow constructs deliberately inserted by the obfuscator rather than actual semantic differences.

Overall, the combination of localized symbolic execution and targeted optimization enables `ROParser` to effectively counter opaque predicates, anti-bruteforce defenses, state explosion, gadget confusion, and indirect branching, achieving near-original CFG structure even under worst-case obfuscation configurations.

Custom Stack Operation Instruction Recovery. The eighth row of Table 1 presents the evaluation results of `ROParser` regarding the recovery of the custom stack operation instructions described in Section 3.2. For dataset D_1, the total number of custom stack operation instructions is 2742. Among these, `ROParser` successfully recovers 2719 instructions, achieving a recovery rate of 99.16%. It indicates that `ROParser` can deobfuscate the vast majority of instructions with high effectiveness. Most of the unrecovered instructions are stack pointer adjustment operations (e.g., *ADD RSP, 0x8*), which are prone to being confused with other stack manipulation instructions during the LLM recognition process due to their semantic similarity. Additionally, Raindrop implements several dedicated instructions to preserve and restore register values, protecting active register contents. These instructions account for only about 0.2% of the obfuscated program and do not carry significant semantic meaning. Since they do not affect core deobfuscation functionality, such instructions are not considered.

Redundant Code Elimination. The ninth row of Table 1 presents the evaluation results of `ROParser` regarding the elimination of redundant instructions described in Section 3.2. For dataset D_2, `ROParser` removes approximately 25.56% of redundant instructions from gadgets. Since ROPfuscator primarily selects single-instruction gadgets, most redundant instructions in this set result from combining multiple instructions. For example, converting sequences like *MOV ECX, ESP; MOV [ECX], ECX;* into a more efficient form such as *MOV [ECX], ESP;*. Most of these redundant instructions are eliminated using the constant propagation algorithm. For dataset D_3, which consists of real-world CVE exploit code ROP chains, the tool removes 15.36% of redundant instructions. Most redundant instructions originate from additional instructions within complex gadgets, with a smaller portion resulting from instruction merging. Eliminating such redundant instructions significantly improves the readability of the deobfuscated ROP code.

Table 3
Execution time of `ROParser` and baseline tools.

Tool	Task	Time (s)
`ROParser`	CFG Recovery	2.52
	Custom Stack Operation	13.32
	Redundant Instruction Eliminate	0.57
ROPdissector	ROP Deobfuscation	1.58
ROPMEMU	ROP Emulation	3.27
	ROP Chain Extract and Shape	7.87

Table 4
Feature comparison of ROP deobfuscation tools.

Ability	`ROParser`	ROPdissector	ROPMEMU
Basic CFG recovery	●	●	●
T1-Opaque-Encoded Branch	●	◍	◍
T2-State Explosion	●	◍	○
T3-Anti-Bruteforce	●	○	○
T4-Gadget Confusion	●	○	◍
T5-Indirect Branch	●	○	○
T6-Custom Stack Operation	●	○	○
T7-Code Redundancy	●	○	○

Full = ●, Partial = ◍, None = ○.

6.3. Performance (RQ2)

We evaluated the performance and runtime efficiency of `ROParser`, focusing on execution time and operational costs, including token consumption during the LLM process.

Table 3 presents the runtime results of `ROParser` and the baseline tools on the dataset D_1-basic. The figure shows the average time each tool spent analyzing different functions in various stages. `ROParser` operates in three stages: CFG recovery, custom stack instruction recovery, and redundant instruction elimination, which take 2.52, 13.32, and 0.57 s on average, respectively. The first and third stages are relatively fast, while the recovery of custom stack instructions requires longer processing time due to network latency and inference delays introduced by the LLM. Despite this, the total time cost remains within an acceptable range. The maximum total analysis time observed is 37.42 seconds, and the minimum is 5.31 s, both falling within an acceptable range. Since the enhanced dataset primarily strengthens control flow obfuscation, its impact on the instruction-level deobfuscation stage is negligible. The execution times of the control flow recovery stage are recorded in the Table 2, showing that even under the most aggressive obfuscation configurations, the additional overhead remains within an acceptable range.

The custom stack instruction recovery process involves the LLM process. The average total token consumption is 1630 tokens per function, with an average input of 1487 tokens and an average output of 143 tokens per function. Based on the current price of the DeepSeek-V3 model, the estimated average cost is approximately $0.0192 per function.

6.4. Comparison (RQ3)

We compared `ROParser` with baselines to highlight the advantages of our approach. As shown in Table 4, our evaluation found that most control flow obfuscation techniques described in the Threat Model successfully break baseline tools, leading to failures such as crashes, excessive execution time, state explosion, and incorrect branch analysis.

For Threat T1, baseline tools rely on emulated execution to explore control flow. While they may occasionally reach the true successor when given favorable inputs, this behavior does not generalize and is not robust to the opaque-encoded branch. For Threat T2, ROPdissector's search strategy can tolerate limited state explosion, whereas ROPMEMU records all execution paths, resulting in prohibitive performance overhead and poor scalability. For Threat T3, both baselines rely on simple EFLAGS flipping to brute-force conditional branch, making them

Table 5
Deobfuscation statistics and performance evaluation on GNU Coreutils.

Binary	Logic Type	Obfuscation Features (Threat)							Deobfuscation Results					
		T1	T2	T3	T4	T5	T6	T7	Gadgets	Instrs	Blocks	Edges	Ana-time	Opt-time
`test`	Boolean Logic	✓	✓	✓	✓	×	✓	✓	901	50	6	6	4.152	3.395
`whoami`	System State Query	✓	✓	✓	✓	×	✓	✓	1556	84	9	7	6.674	4.489
`date`	Format Parsing	×	×	✓	✓	×	✓	×	1799	549	105	164	25.626	121.448
`dirname`	String Manipulation	✓	✓	✓	✓	×	✓	✓	2101	119	18	22	20.744	6.899
`cat`	Stream I/O	×	×	✓	✓	×	✓	×	2286	771	170	276	13.350	122.545
`mkdir`	System Side-effects	×	✓	✓	✓	✓	✓	✓	3753	227	40	59	18.100	34.177
`ls`	Directory Traversal	×	×	✓	✓	✓	✓	×	3886	1325	297	498	26.347	85.191
`base64`	Bitwise Arithmetic	×	✓	✓	✓	✓	✓	✓	6269	399	83	128	50.200	55.262

Note: T1: Anti-ROP-disasm; T2: State Explosion; T3: Anti-Bruteforce; T4: Gadget Confusion; T5: Indirect Jumps; T6: Custom Stack-Operation Instructions; T7: Redundant Instructions. (✓: Active; ×: Disabled due to generation constraints.)

incapable of handling Anti-Bruteforce constraints. For Threat T4, ROPdissector depends on externally provided ROP gadget addresses and therefore cannot reliably handle gadget confusion. ROPMEMU may recover misaligned gadgets during emulation. Still, it cannot distinguish between real gadgets and data masquerading as gadgets. For Threat T5, both baselines fail to enumerate all feasible successors of indirect branches, leading to incomplete or fragmented control flow recovery. For Threat T6 and T7, neither baseline has a targeted recovery function.

To preserve functionality across all tools, we perform a fair comparison with baselines using the dataset D_1-basic. The evaluation is conducted across two key areas: the effectiveness of the control flow analysis and the efficiency of the runtime.

CFG Recovery. Fig. 5 shows a comparison of the CFG similarity between `ROParser` and the baseline tools, presenting the CFG similarity sorted among samples in the dataset. `ROParser` achieves an average CFG similarity of 97.97%, outperforming both ROPdissector (91.38%) and ROPMEMU (84.71%). Moreover, in 83% of the analysis results, `ROParser` achieves a perfect 100% similarity score. The baseline tools exhibit lower similarity in most samples, primarily due to their incomplete handling of indirect calls and imprecise recovery of conditional branch edges. For example, ROPMEMU only supports condition handling through the *ZF* flag adjustment, making it unable to accurately process jump instructions such as *JL*, *JGE*, *JS*, and *JNS*, which are not affected by the *ZF* flag. These limitations highlight the superiority of `ROParser` in control flow recovery.

Runtime Efficiency. Table 3 presents a comparison of the runtime performance between `ROParser` and the baseline tools. The figure shows the runtime of each tool by processing stage. As detailed in Section 4, `ROParser` consists of three stages: robust CFG recovery, custom stack instruction recovery, and redundant instruction elimination. ROPdissector includes only a CFG recovery stage, with an average time of 1.58 s. ROPMEMU comprises two stages: ROP emulation and ROP chain extraction and shaping, which take 3.27 and 7.87 s on average, respectively. Due to the application of the LLM, the total runtime of `ROParser` is slightly longer than that of the baseline tools. However, when comparing only the CFG recovery stage, `ROParser` demonstrates certain advantages. More importantly, the additional functionality for custom stack instructions recovery and redundant instructions elimination significantly enhances deobfuscation effectiveness while still operating within an acceptable time range. This balance between performance and capability demonstrates the practical value and superiority of `ROParser`.

6.5. Practicality (RQ4)

To evaluate the scalability and generalizability of our framework on real-world programs, we applied our analysis and optimization pipeline to the datasets D_3 and D_4.

For the CVE ROP chain selection (dataset D_3), the dataset covers a total of 68 CVEs, comprising 1523 gadgets, most of which are used to invoke system calls or privilege escalation functions. The evaluation results show that `ROParser` successfully recovers semantically meaningful instruction sequences and removes redundant instructions with an elimination rate of approximately 15.36%. It demonstrates both the ability to analyze real-world ROP-based exploits and the practicality of eliminating redundant instructions in `ROParser`.

For the GNU Coreutils dataset selection (dataset D_4), these binaries cover a diverse range of logic types, from intensive arithmetic operations to complex control flow structures.

We utilized the Raindrop obfuscator to transform the `main` function of each binary into a ROP payload. We aimed to apply the maximum obfuscation intensity supported by the tool, including *Anti-ROP-disasm*, *State Explosion*, *Anti-bruteforce*, and *Gadget Confusion*.

However, due to inherent constraints within the payload generation tool (Raindrop), specific obfuscation configurations could not be universally applied. Table 5 details the active obfuscation features for each binary.

Specifically, the *Anti-ROP-disasm* feature failed across most of the large samples because the payload generator failed to allocate sufficient registers for obfuscated conditional jumps. Similarly, the *State Explosion* feature was omitted for `ls`, `cat`, and `date` because the generator encountered displacement limitations when translating jump tables. Despite these generation-side limitations, features such as *Anti-bruteforce* and *Gadget Confusion* were successfully applied, ensuring that the generated ROP chains retained a high degree of complexity suitable for benchmarking our analysis framework.

Table 5 presents the quantitative results. The *Gadget* column represents the number of ROP gadgets in the obfuscated payload, while *Instr*, *Block*, and *Edge* represent the count of instructions, basic blocks, and control flow edges in the recovered assembly, respectively.

To verify the correctness of the recovered logic, we patched the deobfuscated assembly code back into the original binaries by creating a new code segment. We executed the rewritten binaries against the official GNU Coreutils regression test suite (make check). All test programs passed the coreutils official test suite, proving that our framework successfully recovered the complete and correct program semantics.

The time consumption is divided into *Ana-time* (LSE & RVSA analysis) and *Opt-time* (LLM-based recovery and optimization). The analysis phase remains efficient (e.g., 50.2s for `base64`) even for large gadget chains, validating the efficiency of our LSE design. The optimization phase consumes more time due to the inference latency of the LLM, but it remains within a practical range for static analysis tasks.

6.6. Robustness (RQ5)

To evaluate the correct rate of the LLM in the custom stack operation instruction recovery process, we evaluated 23 incorrect instructions (of the 2742 instructions evaluated in 6.2) by performing 100 independent recovery attempts per instruction. Among the 2300 trials in total, errors occurred in 113 cases, indicating an average error rate of 4.91% per instruction. This result suggests that the LLM exhibits a relatively low

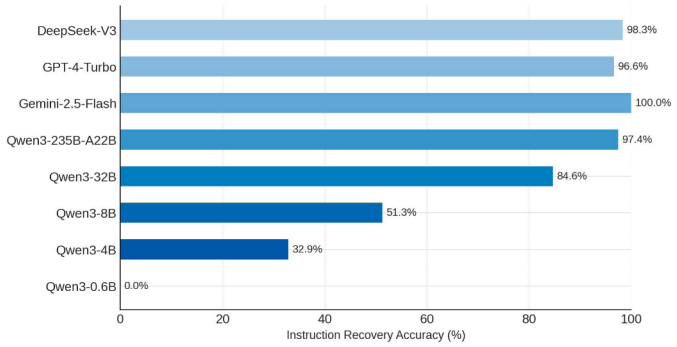


Fig. 6. Robustness evaluation result of `ROParser`.

probability of error during instruction recovery, sufficient to support instruction optimization in the ROP deobfuscation process.

To understand how sensitive our custom stack operation instruction recovery component is to the choice and scale of the LLM, we conducted an ablation evaluation comparing several representative models, including GPT-4-Turbo, DeepSeek-V3, Gemini-2.5-Flash, and the Qwen3 family across a wide range of parameter sizes (235B-A22B/32B/8B/4B/0.6B). Fig. 6 presents the instruction recovery accuracy across selected models.

Overall, the results show a clear correlation between model capacity and recovery effectiveness. Large-scale commercial models such as Gemini-2.5-Flash, DeepSeek-V3, Qwen3, and GPT-4-Turbo demonstrate the strongest performance, recovering nearly all obfuscated instructions with very few errors. Mid-scale models such as Qwen3-32B and Qwen3-8B achieve moderate accuracy. Meanwhile, small models like Qwen3-4B and Qwen3-0.6B exhibit a sharp performance drop, failing to recover a substantial portion of the instruction semantics.

This trend shows that accurate instruction recovery benefits from both high-quality model training and sufficient parameter capacity, which large-scale commercial models naturally provide, thereby achieving the highest recovery accuracy. At the same time, our ablation study demonstrates that `ROParser` itself is not tied to any specific model: the recovery process remains functional across all evaluated LLMs, and its accuracy scales with model size. It indicates that `ROParser` can take full advantage of stronger future LLMs when available, while still supporting deployment in resource-constrained environments by sacrificing some accuracy for efficiency when necessary.

7. Discussion

Robustness Against Future Obfuscation. While our approach demonstrates strong effectiveness against a wide range of existing ROP obfuscation techniques, future obfuscators may introduce targeted countermeasures, for example, by modifying the structure of custom stack operations. Such adaptations may increase the difficulty of analysis. Nevertheless, our framework is fundamentally based on few-shot prompting, which remains flexible: adapting the prompts or supplying additional examples can naturally extend the system to new instruction patterns without redesigning the overall methodology.

Dependence on LLM Capability. Our LLM-based instruction recovery is affected by both model quality and parameter scale. Smaller models exhibit lower instruction recovery accuracy, as shown in our ablation results. However, modern commercial-scale models consistently provide sufficient reasoning capacity to support high-accuracy instruction reconstruction. Thus, while model choice influences performance, it does not pose a practical limitation given the availability of strong models. Additionally, due to network latency and model inference time, the analysis duration is prolonged but remains within an acceptable range.

Complementarity with Existing Techniques. Our deobfuscation framework is orthogonal to many existing approaches to code simplification, such as opaque predicate elimination, MBA simplification, and other instruction deobfuscation techniques (Tung and Harris, 2020; Liu et al., 2021; Mariano et al., 2024; Roh et al., 2025). These methods can be applied before or after our analysis to further improve code quality. This compatibility highlights that our control-flow-centric design complements, rather than replaces, current advances in program deobfuscation.

Future Work. Beyond ROP obfuscation, extending our localized symbolic execution and reduced value-set analysis to other control flow obfuscation is a promising direction. In addition, future work will further expand the capability of LLM-driven few-shot instruction recovery, particularly in understanding how well prompts generalize to previously unseen custom instructions and how robust the model remains under adversarially crafted obfuscation patterns. Advancing this direction would contribute to building more adaptable and broadly applicable deobfuscation frameworks.

8. Related work

ROP also serves as an effective technique to enhance antivirus evasion capabilities. ROPInjector (Ntantogian et al., 2019) transforms shellcode or malware into equivalent ROP chains to bypass detection. ROPNeedle (Borrello et al., 2019) implements malware trigger conditions using ROP and encrypts the ROP chains to protect malware from antivirus software.

ROP can be effectively applied in non-attack scenarios. Ropsteg (Lu et al., 2014) implements binary program steganography through ROP chains. Parallax (Andriesse et al., 2015) generates ROP chains from gadgets in target binary files to verify code integrity by executing the ROP chain.

Symbolic Execution. Symbolic execution can be used not only in ROP deobfuscation but also in various deobfuscation scenarios (Boyer et al., 1975). For Virtualization Obfuscation (Salwan et al., 2018; Shoshitaishvili et al., 2016), symbolic execution can track virtual machine instruction execution flows to recover the original program logic. For Control Flow Flattening techniques (Kan et al., 2019; Dong et al., 2022), symbolic execution enables the reconstruction of original control flow structures by analyzing dispatcher behavior and state variable changes. For opaque predicates (Ming et al., 2015; Bardin et al., 2017), symbolic execution can determine condition authenticity through constraint solving, effectively identifying and eliminating false control flow branches.

Compiler Optimization. Compiler optimization, such as constant propagation and liveness analysis, is frequently used in program deobfuscation. Yadegari et al. (2015) employ the constant folding method in trace simplification to recover control flow. Liang et al. (2018) address virtualization-based obfuscation by lifting virtual instructions into high-level code and leveraging standard compiler optimizations.

Large Language Model. With the widespread use of LLMs, some studies have experimented with LLM obfuscation and deobfuscation (Jelodar et al., 2025). Recent studies demonstrate that LLMs can effectively perform source code obfuscation (Mohseni et al., 2025; Lin et al., 2024), primarily aimed at protecting sensitive information and preventing privacy leakage during software development processes assisted by LLMs. For deobfuscation, LLMs can enhance disassembly effectiveness for obfuscated executables by leveraging powerful semantic analysis capabilities (Rong et al., 2024). Through LLM training techniques such as fine-tuning (Choi et al., 2024) and pre-training (Lachaux et al., 2021), LLMs have demonstrated effectiveness in directly addressing various deobfuscation challenges.

9. Conclusion

In this paper, we present `ROParser`, a novel ROP deobfuscation framework. `ROParser` addresses three key challenges in ROP deobfuscation: robust control flow recovery, custom stack operation instruction

deobfuscation, and redundant instruction optimization. `ROParser` employs three key techniques to address these key challenges: localized symbolic execution combined with reduced value set analysis, few-shot learning with LLM, and a semantics-guided redundant code optimization strategy.

`ROParser` has been evaluated across four datasets and compared with state-of-the-art methods, including ROPdissector and ROPMEMU. Our evaluation results demonstrate that `ROParser` outperforms previous deobfuscation approaches in terms of functionality and is highly effective in deobfuscation. The practicality of `ROParser` is highlighted by its potential applications in ROP program deobfuscation and the analysis of ROP-based malicious payloads.

CRediT authorship contribution statement

Haoran Liu: Writing – review & editing, Writing – original draft, Software, Resources, Project administration, Methodology, Investigation, Formal analysis, Data curation, Conceptualization; **Tieming Liu:** Writing – review & editing, Supervision, Conceptualization; **Rui Chang:** Writing – review & editing; **Jian Lin:** Writing – review & editing, Supervision, Methodology, Conceptualization; **Zuozheng Zhou:** Data curation; **Jing Jing:** Writing – review & editing, Supervision.

Data availability

Data will be made available on request.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

This work was supported by the Natural Science Foundation of Henan Province of China (No. 242300420698).

References

Andriesse, D., Bos, H., Slowinska, A., 2015. Parallax: implicit code integrity verification using return-oriented programming. In: 2015 45th Annual IEEE/IFIP International Conference on Dependable Systems and Networks. IEEE, pp. 125–135.

Angelini, M., Blasilli, G., Borrello, P., Coppa, E., D'Elia, D.C., Ferracci, S., Lenti, S., Santucci, G., 2018. Ropmate: visually assisting the creation of rop-based exploits. In: 2018 IEEE Symposium on Visualization for Cyber Security (VizSec). IEEE, pp. 1–8.

Banescu, S., Collberg, C., Ganesh, V., Newsham, Z., Pretschner, A., 2016. Code obfuscation against symbolic execution attacks. In: Proceedings of the 32nd Annual Conference on Computer Security Applications, pp. 189–200.

Bardin, S., David, R., Marion, J.-Y., 2017. Backward-bounded DSE: targeting infeasibility questions on obfuscated codes. In: 2017 IEEE Symposium on Security and Privacy (SP). IEEE, pp. 633–651.

Blazytko, T., Contag, M., Aschermann, C., Holz, T., 2017. Syntia: synthesizing the semantics of obfuscated code. In: 26th USENIX Security Symposium (USENIX Security 17), pp. 643–659.

Bletsch, T., Jiang, X., Freeh, V.W., Liang, Z., 2011. Jump-oriented programming: a new class of code-reuse attack. In: Proceedings of the 6th ACM Symposium on Information, Computer and Communications Security, pp. 30–40.

Borrello, P., Coppa, E., D'Elia, D.C., Demetrescu, C., 2019. The ROP needle: hiding trigger-based injection vectors via code reuse. In: Proceedings of the 34th ACM/SIGAPP Symposium on Applied Computing, pp. 1962–1970.

Borrello, P., Coppa, E., D'Elia, D.C., 2021. Hiding in the particles: when return-oriented programming meets program obfuscation. In: 2021 51st Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN). IEEE, pp. 555–568.

Boyer, R.S., Elspas, B., Levitt, K.N., 1975. Select-a formal system for testing and debugging programs by symbolic execution. ACM SigPlan Notices 10 (6), 234–245.

Choi, B., Jin, H., Lee, D.H., Choi, W., 2024. ChatDEOB: an effective deobfuscation method based on large language model. In: International Conference on Information Security Applications. Springer, pp. 151–163.

Cloosters, T., Paaßen, D., Wang, J., Draissi, O., Jauernig, P., Stapf, E., Davi, L., Sadeghi, A.-R., 2022. Riscyrop: automated return-oriented programming attacks on risc-v and arm64. In: Proceedings of the 25th International Symposium on Research in Attacks, Intrusions and Defenses, pp. 30–42.

Collberg, C., Thomborson, C., Low, D., 1998. Manufacturing cheap, resilient, and stealthy opaque constructs. In: Proceedings of the 25th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, pp. 184–196.

De Pasquale, G., Nakanishi, F., Ferla, D., Cavallaro, L., 2023. Ropfuscator: robust obfuscation with rop. In: 2023 IEEE Security and Privacy Workshops (SPW). IEEE, pp. 1–10.

D'Elia, D.C., Coppa, E., Salvati, A., Demetrescu, C., 2019. Static analysis of ROP code. In: Proceedings of the 12th European Workshop on Systems Security, pp. 1–6.

CEA-SEC, 2012. miasm. GitHub repository. https://github.com/cea-sec/miasm.

Dong, W., Lin, J., Chang, R., Wang, R., 2022. CadeCFF: compiler-agnostic deobfuscator of control flow flattening. In: Proceedings of the 13th Asia-Pacific Symposium on Internetware, pp. 282–291.

Follner, A., Bartel, A., Bodden, E., 2016. Analyzing the gadgets: towards a metric to measure gadget quality. In: Engineering Secure Software and Systems: 8th International Symposium, ESSoS 2016, London, UK, April 6–8, 2016. Proceedings 8. Springer, pp. 155–172.

Google, 2024. kernel-research. GitHub repository. https://github.com/google/kernel-research.

Graziano, M., Balzarotti, D., Zidouemba, A., 2016. Ropmemu: a framework for the analysis of complex code-reuse attacks. In: Proceedings of the 11th ACM on Asia Conference on Computer and Communications Security, pp. 47–58.

Huang, X., Zhang, L.L., Cheng, K.-T., Yang, F., Yang, M., 2023. Fewer is more: boosting LLM reasoning with reinforced context pruning. arXiv preprint arXiv:2312.08901.

Jelodar, H., Bai, S., Hamedi, P., Mohammadian, H., Razavi-Far, R., Ghorbani, A., 2025. Large language model (LLM) for software security: Code analysis, malware analysis, reverse engineering. arXiv preprint arXiv:2504.07137.

Kan, Z., Wang, H., Wu, L., Guo, Y., Xu, G., 2019. Deobfuscating android native binary code. In: 2019 IEEE/ACM 41st International Conference on Software Engineering: Companion Proceedings (ICSE-Companion). IEEE, pp. 322–323.

Lachaux, M.-A., Roziere, B., Szafraniec, M., Lample, G., 2021. Dobf: a deobfuscation pre-training objective for programming languages. Adv. Neural Inf. Process. Syst. 34, 14967–14979.

Liang, M., Li, Z., Zeng, Q., Fang, Z., 2018. Deobfuscation of virtualization-obfuscated code through symbolic execution and compilation optimization. In: Information and Communications Security: 19th International Conference, ICICS 2017, Beijing, China, December 6–8, 2017, Proceedings 19. Springer, pp. 313–324.

Lin, Y., Wan, C., Fang, Y., Gu, X., 2024. Codecipher: learning to obfuscate source code against LLMs. arXiv preprint arXiv:2410.05797.

Liu, A., Feng, B., Xue, B., Wang, B., Wu, B., Lu, C., Zhao, C., Deng, C., Zhang, C., Ruan, C., et al., 2024. Deepseek-v3 technical report. arXiv preprint arXiv:2412.19437.

Liu, B., Shen, J., Ming, J., Zheng, Q., Li, J., Xu, D., 2021. {MBA-Blast}: unveiling and simplifying mixed {Boolean-Arithmetic} obfuscation. In: 30th USENIX Security Symposium (USENIX Security 21), pp. 1701–1718.

Lu, K., Xiong, S., Gao, D., 2014. Ropsteg: program steganography with return oriented programming. In: Proceedings of the 4th ACM Conference on Data and Application Security and Privacy, pp. 265–272.

Lu, K., Zou, D., Wen, W., Gao, D., 2011. Derop: removing return-oriented programming from malware. In: Proceedings of the 27th Annual Computer Security Applications Conference, pp. 363–372.

Mariano, B., Wang, Z., Pailoor, S., Collberg, C., Dillig, I., 2024. Control-flow deobfuscation using trace-informed compositional program synthesis. Proceedings of the ACM on Programming Languages 8 (OOPSLA2), 2211–2241.

Ming, J., Xu, D., Wang, L., Wu, D., 2015. Loop: logic-oriented opaque predicate detection in obfuscated binary code. In: Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security, pp. 757–768.

Mohseni, S., Mohammadi, S., Tilwani, D., Saxena, Y., Ndawula, G.K., Vema, S., Raff, E., Gaur, M., 2025. Can LLMs obfuscate code? a systematic analysis of large language models into assembly code obfuscation. In: Proceedings of the AAAI Conference on Artificial Intelligence. Vol. 39, pp. 24893–24901.

Mu, D., Guo, J., Ding, W., Wang, Z., Mao, B., Shi, L., 2018. Ropob: obfuscating binary code via return oriented programming. In: Security and Privacy in Communication Networks: 13th International Conference, SecureComm 2017, Niagara Falls, on, Canada, October 22–25, 2017, Proceedings 13. Springer, pp. 721–737.

Ntantogian, C., Poulios, G., Karopoulos, G., Xenakis, C., 2019. Transforming malicious code to ROP gadgets for antivirus evasion. IET Inf. Secur. 13 (6), 570–578.

De Sutter, B., Schrittwieser, S., Coppens, B., Kochberger, P., 2024. Evaluation Methodologies in Software Protection Research. ACM Comput. Surv. 57 (4), 1–41.

Rapid7, 2011. metasploit-framework. GitHub repository. https://github.com/rapid7/metasploit-framework.

Roh, Y., Paik, J.-Y., Kwon, J., Cho, E.-S., 2025. gMBA: expression semantic guided mixed boolean-arithmetic deobfuscation using transformer architectures. In: Findings of the Association for Computational Linguistics: ACL 2025, pp. 21882–21888.

Rong, H., Duan, Y., Zhang, H., Wang, X., Chen, H., Duan, S., Wang, S., 2024. Disassembling obfuscated executables with LLM. arXiv preprint arXiv:2407.08924.

Salwan, J., Bardin, S., Potet, M.-L., 2018. Symbolic deobfuscation: from virtualized code back to the original. In: International Conference on Detection of Intrusions and Malware, and Vulnerability Assessment. Springer, pp. 372–392.

Schuster, F., Tendyck, T., Liebchen, C., Davi, L., Sadeghi, A.-R., Holz, T., 2015. Counterfeit object-oriented programming: on the difficulty of preventing code reuse attacks in c++ applications. In: 2015 IEEE Symposium on Security and Privacy. IEEE, pp. 745–762.

Shacham, H., 2007. The geometry of innocent flesh on the bone: return-into-libc without function calls (on the x86). In: Proceedings of the 14th ACM Conference on Computer and Communications Security, pp. 552–561.

Shoshitaishvili, Y., Wang, R., Salls, C., Stephens, N., Polino, M., Dutcher, A., Grosen, J., Feng, S., Hauser, C., Kruegel, C., et al., 2016. Sok:(state of) the art of war: offensive techniques in binary analysis. In: 2016 IEEE Symposium on Security and Privacy (SP). IEEE, pp. 138–157.

Taki, Y., Fukumitsu, M., Yumura, T., 2024. Experiments on ROP attack with various instruction set architectures. Bull. Netw. Comput. Syst. Softw. 13 (1), 23–28.

TUM-i4, 2015. obfuscation-benchmarks. GitHub repository. https://github.com/tum-i4/obfuscation-benchmarks.

Tung, Y.-J., Harris, I.G., 2020. A heuristic approach to detect opaque predicates that disrupt static disassembly. In: Proceedings 2020 Workshop on Binary Analysis Research (BAR'20). Available at: https://www.ndss-symposium.org/ndss-paper/auto-draft-89/.

Udupa, S.K., Debray, S.K., Madou, M., 2005. Deobfuscation: reverse engineering obfuscated code. In: 12th Working Conference on Reverse Engineering (WCRE'05). IEEE, pp. 10–pp.

Yadegari, B., Johannesmeyer, B., Whitely, B., Debray, S., 2015. A generic approach to automatic deobfuscation of executable code. In: 2015 IEEE Symposium on Security and Privacy. IEEE, pp. 674–691.

Zhong, N., Chen, Y., Zou, Y., Xing, X., Dong, J., Xian, B., Zhao, J., Li, M., Liu, B., Huo, W., 2024. Tgrop: top gun of return-oriented programming automation. In: European Symposium on Research in Computer Security. Springer, pp. 130–152.